Data Modeling và Resource Design

Thiết kế resource structure · Quan hệ dữ liệu · Pagination · Filtering & Sorting

Resource Design | Data Modeling | Relationships | Pagination | Filtering

Resource Hierarchy

```
GET     /books
POST    /books
GET     /books/{id}
GET     /books/{id}/reviews
GET     /authors/{id}/books
```

Pagination Response

```
"data": [...],
"pagination": {
  "cursor": "eyJpZCI6NTB9",
  "has_more": true
}
```

Filtering & Sorting

```
/books?genre=tech
&year=2023
&sort=price&order=asc
&limit=20&after=abc
```

Mục tiêu

Thiết kế resource structure

Dựa trên sự hiểu biết về mối quan hệ giữa **data model** và **API design**, thiết kế resource structure cho miền dữ liệu cụ thể.

Resource Naming | Relationships | Hierarchy

So sánh pagination strategies

Phân tích ưu/nhược điểm và ứng dụng thực tế của từng chiến lược phân trang trong API.

Offset | Cursor-based | Keyset

Tạo data model phù hợp với business requirements

Ánh xạ business domain sang API resources và viết Flask code hoàn chỉnh.

Flask | SQLAlchemy | JSON Schema

Nội Dung Buổi Học

1. **Resource là gì? Data model vs API model**
   Định nghĩa resource, phân biệt DB model với API representation

2. **Naming Conventions & Resource Relationships**
   Quy tắc đặt tên endpoint, quan hệ 1-N, N-M, nested resources trong URL

3. **Resource Representation & JSON Schema**
   Cấu trúc JSON response, field selection, schema validation, Flask + SQLAlchemy lab

4. **Pagination Strategies**
   Offset, cursor-based, keyset pagination — so sánh hiệu năng & trade-off thực tế

5. **Filtering, Sorting & Tổng kết**
   Query parameter patterns, filtering design, sorting best practices, bài tập về nhà

Tại sao Data Modeling quan trọng?

Một API với resource design tốt sẽ **giảm friction** cho developer consumer, **dễ versioning** và bảo trì về sau. Ngược lại, một API được thiết kế vội vàng theo DB schema sẽ lộ ra *implementation detail* — gây khó khăn khi refactor.

Bài học từ Stripe API

Stripe thiết kế resource `Customer`, `PaymentIntent`, `Invoice` độc lập với DB schema. Các resource công khai không nhất thiết phản ánh cấu trúc bảng nội bộ.

Bài học từ Twitter (X) API v2

Twitter API v1 trả về toàn bộ user object trong mỗi tweet (dư thừa). API v2 dùng `expansions` để tách resource — giúp client chọn dữ liệu liên quan khi cần.

Vấn đề thực tiễn hôm nay

Chúng ta sẽ xây dựng một Bookstore API hoàn chỉnh từ domain model đến pagination bằng Flask, áp dụng đúng các nguyên tắc trên.

Resource là gì?

Resource (tài nguyên) là một đối tượng dữ liệu có **identity**, có **state** và có thể **được biểu diễn** qua URI trong API.

> **Định nghĩa của Roy Fielding (2000)**
>
> *"Any information that can be named can be a resource: a document or image, a temporal service, a collection of other resources, a non-virtual object (e.g. a person), and so on."*
>
> — Roy Fielding, ch.5 Architectural Styles and the Design of Network-based Software Architectures

3 tính chất của một Resource

1. Identity (Định danh)

Resource có thể được định danh bằng URI; nên ưu tiên URI ổn định và nhất quán.

```
/users/42
/orders/2024-001
/books/978-0-13-468599-1
```

2. State (Trạng thái)

Resource có dữ liệu bên trong thay đổi theo thời gian. Client chỉ "nhìn" state tại thời điểm truy cập.

```
"available": true,
"stock": 14,
"price_vnd": 250000
```

3. Representation

Representation có thể ở dạng JSON, XML hoặc định dạng khác theo media type và API contract.

```
{ "id": 42,
"name": "An",
"role": "admin" }
```

Ví dụ: Resource trong hệ thống Bookstore

Resource	URI mẫu
Book	`/books/42`
Author	`/authors/15`
Review (sub-resource)	`/books/42/reviews/8`
Order	`/orders/ord_2024_001`
User Cart	`/users/42/cart`

> ⚠ **Lưu ý**
>
> Resource ≠ đối tượng database. Một resource có thể map từ nhiều bảng, hoặc chỉ là một phần của bảng.

> **Sai lầm phổ biến:** thiết kế resource theo bảng database (vd: `/tbl_users`, `/tbl_orders`) thay vì theo business domain.
>
> → Khi DB schema thay đổi, API cũng phải đổi theo — đây là **leaky abstraction**.

Phân biệt Resource, Domain Model và Database Schema

API design tốt là API **tách biệt với** DB schema, chỉ lấy những gì consumer cần.

LỚP 1 · PERSISTENCE

DB Schema

Mô tả **cách lưu trữ vật lý**: bảng, cột, FK, index. Có thể chuẩn hoá (3NF/BCNF) và tối ưu cho query performance.

```
CREATE TABLE users (
  id BIGSERIAL PRIMARY KEY,
  email VARCHAR(255)
    UNIQUE NOT NULL,
  pwd_hash BYTEA NOT NULL,
  full_name VARCHAR(120),
  created_at TIMESTAMPTZ
);
```

Đặc điểm: thiên về implementation, phụ thuộc DB engine (Postgres, MySQL, ...)

LỚP 2 · DOMAIN

Domain Model

Mô tả **thực thể nghiệp vụ** và quan hệ giữa chúng — độc lập DB, thường dùng trong OOP/DDD. Ánh xạ sang nhiều bảng nếu cần.

```
from dataclasses import dataclass
from datetime import datetime

@dataclass
class User:
  id: int
  email: str
  profile: Profile      # 1-1
  orders: list[Order]   # 1-N
```

Đặc điểm: business logic thuần, dễ test, có thể ánh xạ sang nhiều DB hoặc không cần DB.

LỚP 3 · API

API Resource

Mô tả **cái client nhìn thấy**: URIs, methods, JSON fields. Một resource có thể được gộp từ nhiều entity, computed fields, hoặc drop sensitive fields.

```
{
  "id": 42,
  "display_name": "An Nguyen",
  "avatar_url": "...",
  // NO email, NO pwd_hash
  // created_at không được expose
  // joined từ bảng profiles
}
```

Đặc điểm: contract với consumer, ổn định qua version, response format tuỳ content-type.

DB Schema (1 user → 1 row) ≠ **Domain Model** (1 user → kèm Profile, kèm Orders ảo) ≠ **API Resource** (những gì client cần thấy)

Quy trình thiết kế resource từ requirement

Áp dụng cho **mọi domain**: từ LMS, e-commerce, banking, IoT — đầu vào là business requirements, đầu ra là resource set hoàn chỉnh.

BƯỚC 1
Xác định Entities

- Đọc requirements, đánh dấu **danh từ** (user, order, book)
- Phân biệt entities vs value objects (address, role)
- Tìm events chính để suy ra entities phái sinh (OrderCreated → Order)
- Hỏi stakeholder: *"Đối tượng nào có lifecycle riêng?"*

Output: Danh sách entity ứng viên

›

BƯỚC 2
Đặt tên Resource

- Resource là **danh từ số nhiều**, viết thường, dấu gạch ngang
- Method là động từ: GET/POST/PUT/DELETE
- Tránh động từ trong URL: `/getUsers`
- Tránh lộ DB: `/tbl/users`

Output: Bảng URI → Method

›

BƯỚC 3
Phân tích Quan hệ

- Vẽ ER diagram — đánh dấu cardinality
- Phân biệt composition (1:N) vs association (N:M)
- Quyết định nested URL vs top-level URL
- Tránh nested sâu quá 2 cấp

Output: ERD + URL hierarchy

›

BƯỚC 4
Định nghĩa Schema

- Liệt kê JSON fields cho từng resource
- Ghi rõ type, format, required/optional
- Xác định field nào **không** expose
- Validate bằng JSON Schema hoặc Pydantic

Output: JSON Schema + sample response

Mẹo nhanh khi đặt tên resource

Câu hỏi 1

Entity này có **tồn tại độc lập** (vd: User, Order) hay chỉ là thuộc tính (vd: Address)?

Câu hỏi 2

Resource có cần **collection endpoint** trả danh sách (vd: /books) hay chỉ single-item?

Câu hỏi 3

Resource này có thể được **truy cập trực tiếp** qua URI riêng, không cần parent?

Từ requirements → ER diagram → Resource list

Yêu cầu nghiệp vụ

- Sách online — xem, đánh giá, mua
- Sách có **nhiều tác giả**, **1 thể loại**
- Khách hàng có **lịch sử đơn hàng**
- Đơn hàng chứa **nhiều sách**; mỗi sách kèm **SL**
- Khách đánh giá **1–5 sao**
- Tác giả xem DS sách của họ

Actors: PM, khách hàng, tác giả

7 quy tắc đặt tên resource & URL

1. Danh từ số nhiều, dùng kebab-case

`/books` `/users` `/order-items`

`/Book` `/getUser` `/orderItems`

2. Resource dùng HTTP method — không nhúng vào URL

`DELETE /books/42`

`/deleteBook?id=42`

3. ID ổn định — chính sách cấp ID nhất quán

`/orders/ord_2024_00123` `/users/u_42`

`/orders/45 — cần cân nhắc ID tuần tự`

4. Singular chỉ dùng nếu thực sự chỉ có một

`GET /config (system config)`

`/user/{id}`

5. Tránh "vội vàng mapping theo DB schema"

Đừng lộ tên bảng. Chỉ tách riêng `/user_emails`, `/user_phones` khi có nhu cầu nghiệp vụ; cân nhắc field hoặc sub-resource.

6. URL không chứa file extension

`/books.json` `/books?format=xml`

Dùng `Accept` header thay vì extension (recommended) — nhưng extension cũng hợp lệ cho content negotiation.

7. Không dùng động từ cho resource — method đã là động từ

`GET /search?query=...` // OK vì resource hợp lệ

`/login` `/searchBook` // cần đánh giá theo semantics

Ví dụ thực tế — GitHub REST API (một benchmark phổ biến)

Endpoint	Mục đích	Method
/repos/{owner}/{repo}/issues	List issues của repo	GET
/repos/{owner}/{repo}/issues/{number}	Xem 1 issue	GET
/user/repos	Repos của current user	GET
/gists/public	Public gists	GET

Quan hệ One-to-Many: Sub-resource vs. Top-level

Khi nào dùng `/books/42/reviews`? Khi nào tách thành `/reviews?book_id=42`?

> **Câu hỏi quyết định: child có tồn tại độc lập với parent không?**
> **"Phụ thuộc cứng" (strong containment)** — không thể tồn tại nếu parent bị xoá. Review chỉ tồn tại khi Book còn. → **dùng nested URL**.
> **"Phụ thuộc lỏng" (weak reference)** — có thể tồn tại độc lập, có URI riêng. Comment có thể reply comment khác. → **dùng top-level URL** + filter.

Pattern A — Nested (URI chứa parent)

Dùng khi child **không có ý nghĩa** nếu tách khỏi parent. Phù hợp với navigation mental model.

```
GET    /books/42/reviews      // list
POST   /books/42/reviews      // tạo review cho book 42
GET    /books/42/reviews/8    // 1 review cụ thể
PUT    /books/42/reviews/8    // sửa (chỉ owner)
DELETE /books/42/reviews/8    // xoá (chỉ owner)
```

✓ Ưu điểm
- Client navigate rõ ràng
- Thuận tiện áp dụng policy theo parent
- Thể hiện rõ ngữ cảnh parent trong URI

✗ Nhược điểm
- URL dài khi nesting sâu
- Khó cross-reference (vd: tìm tất cả review của user → phải loop qua từng book)

Pattern B — Top-level với filter

Dùng khi child **là first-class resource** — có thể query đa chiều. Phù hợp khi có nhiều cách "view" cùng dữ liệu.

```
GET    /reviews?book_id=42    // review của book 42
GET    /reviews?user_id=15    // review của user 15
GET    /reviews?rating=1      // filter theo sao
GET    /reviews/8             // 1 review riêng lẻ
PUT    /reviews/8             // sửa
DELETE /reviews/8             // xoá
```

✓ Ưu điểm
- Linh hoạt cross-reference
- URL ngắn — dễ cache theo pattern
- Dễ build aggregation endpoint (vd: dashboard)

✗ Nhược điểm
- Filter có thể bị client lạm dụng
- Authorization phức tạp hơn (kiểm tra cả parent và child)

Quan hệ N:M — 4 cách thiết kế

Cả hai bên đều **first-class**. Cần **junction resource** hoặc **nested + pair ID**.

Ví dụ: Book ↔ Author (N:M)

Cách 1 · Filter top-level

```
GET /books?author=15
GET /authors?book=42
```

✓ Đơn giản
✗ Không có per-pair data

Cách 2 · Sub-resource

```
GET /books/42/authors
GET /authors/15/books
```

✓ Navigation rõ
✗ Tạo pair khó

Cách 3 · Junction resource

```
GET /book-authors
POST {book:42,author:15}
DELETE /book-authors/123
```

✓ Mỗi pair có URI riêng

Cách 4 · Nested + pair

```
GET /books/42/authors
POST {author_id:15,role:"co"}
DELETE /books/42/authors/15
```

✓ Hybrid: phổ biến

N:M - Association: Junction có metadata và triển khai Nested + pair

Cả hai bên đều **first-class**. Cần **junction resource** hoặc **nested + pair ID**.

Khi nào cần junction có per-pair data?

Khi cặp A–B có metadata riêng, không thuộc riêng A hay B.

- Author-of-Book: role
- Student-in-Course: grade, status
- User-likes-Post: reaction_type
- Product-in-Order: qty, price

Flask — Cách 4 (Nested + pair) — POST + GET

```
@bp.post("/books/<int:bid>/authors")
def add_author(bid):
  req = request.get_json(silent=True) or {}
  if "author_id" not in req:
    abort(400)
  p = BookAuthor(
    book_id=bid, author_id=req["author_id"],
    role=req.get("role", "co"))
  db.session.add(p); db.session.commit()
  return p.to_dict(), 201

@bp.get("/books/<int:bid>/authors")
def list_authors(bid):
  rows = (db.session.query(Author, BookAuthor.role)
    .join(Author, Author.id == BookAuthor.author_id)
    .filter(BookAuthor.book_id == bid).all())
  return jsonify([
    {"id": a.id, "name": a.name, "role": r}
    for a, r in rows])

# DELETE cùng pattern:
# filter_by(book_id=bid, author_id=aid).delete(); commit(); 204
```

Quyết định khi nào URL lồng nhau, khi nào tách phẳng

Cây quyết định

Q1. Child có tồn tại khi parent bị xoá?

- **Không** → **Cân nhắc nested resource**
- **Có** → Q2

Q2. Child có bao nhiêu context chỉ từ parent?

- **Gần như tất cả** → Nest + show parent context trong response
- **Không / ít** → Top-level + filter

Q3. Có N:M quan hệ với N pair có metadata?

- **Có** → Junction resource + per-pair endpoint
- **Không** → **Cân nhắc filter hoặc liên kết trực tiếp**

Q4. URL có bị sâu quá 2 cấp không?

- **Có (>/a/b/c)** → Refactor: rút ngắn, dùng top-level
- **Không (≤2 cấp)** → OK

Áp dụng cho Bookstore API

Quan hệ	Quyết định	Lý do
Book → Review (1:N)		
Book ↔ Author (N:M)		
Order → OrderItem (1:N)		
User → Order (1:N)		
Book → Category (N:1)		
User → Cart (1:1)		

⚠ Tránh các pattern sau

Quá sâu: `/users/42/orders/abc/items/xyz/reviews` — rút xuống `/reviews?order_item_id=xyz`

Lộ DB: `/book_authors` thay vì `/books/42/authors`

Verbs trong URL: `/books/42/addReview` thay vì `POST /books/42/reviews`

Thiết kế JSON representation từ resource

JSON là **wire format**, không phải data model. Thiết kế để client dùng dễ — không khớp DB.

① Chọn nhất quán quy ước JSON keys

```
{ "firstName": "An",
  "lastLoginAt": "...",
  "isAdmin": false }
```

② ISO 8601 cho date/time

```
"createdAt":
  "2026-08-30T14:30:00Z"
"expiresAt":
  "2027-01-01T00:00:00+07:00"
```

UTC kết thúc bằng `Z`; luôn kèm TZ.

③ Embed related thay vì ID

```
"author": {
  "id": 15, "name": "An"
}
```

Tiết kiệm round-trip khi cần hiển thị.

④ Dùng ID cho collection lớn

```
"categoryIds": [3, 7]
// ?expand=categories
```

Cân nhắc kích thước và nhu cầu client.

⑤ Prefix ID theo resource type

```
"id": "book_42"
"id": "ord_2024_001"
"id": "user_u_15"
```

Một quy ước có thể dùng; không bắt buộc.

⑥ Không lộ field nội bộ

```
// KHÔNG trả
"passwordHash": "...",
"internalId": 4729
```

Whitelist serializer — drop sensitive fields.

❌ Sai — trả về raw DB row

```
{
  "book_id": 42, "book_title": "Clean Code",
  "book_isbn_13": "9780132350884",
  "cat_id": 7, "created_ts": 1725014400,
  "is_deleted": 0
}
```

Vấn đề: lộ chi tiết lưu trữ, field nội bộ; quy ước tên cần nhất quán.

✓ Đúng — API representation

```
{
  "id": "book_42", "title": "Clean Code",
  "isbn": "9780132350884",
  "category": {"id": 7, "slug": "tech"},
  "authors": [{"id": 8, "name": "R.C. Martin",
               "role": "lead"}],
  "price": {"amount": 450000, "currency": "VND"},
  "inStock": true, "publishedAt": "2008-08-01",
  "createdAt": "2026-01-15T08:00:00Z"
}
```

camelCase · ISO 8601 · embed category+author · price có currency

JSON Schema validation: schema cho Book

JSON Schema Draft 2020-12 (json-schema.org). Dùng cho input validation, doc generation, mock data

Schema cho Book

```
{
  "type": "object",
  "required": ["id", "title", "price"],
  "properties": {
    "id": {"type": "string", "pattern": "^book_[a-z0-9]+$"},
    "title": {"type": "string", "minLength": 1},
    "isbn": {"type": "string", "pattern": "^\\d{13}$"},
    "price": {
      "type": "object",
      "properties": {
        "amount": {"type": "integer", "minimum": 0},
        "currency": {"enum": ["VND", "USD", "EUR"]}
      }
    },
    "authors": {
      "type": "array", "minItems": 1,
      "items": {"$ref": "#/$defs/author"}
    }
  },
  "$defs": {
    "author": {
      "type": "object",
      "properties": {
        "id": {"type": "integer"}, "name": {"type": "string"},
        "role": {"enum": ["lead", "co", "ed"]}
      }
    }
  }
}
```

JSON Schema keywords

- type — string / integer / number / boolean / array / object / null
- required — array các field bắt buộc
- enum / pattern — giới hạn / regex
- minimum / maximum — range cho number
- $ref — tham chiếu sub-schema (DRY)

JSON Schema validation trong Python

JSON Schema Draft 2020-12 (json-schema.org). Dùng cho input validation, doc generation, mock data

Validate Flask + jsonschema

```
from jsonschema import validate, ValidationError
import json
SCHEMA = json.load(open("schemas/book.json"))

@bp.post("")  # Blueprint: /api/v1/books
def create_book():
  payload = request.get_json(silent=True)
  try:
    validate(payload, SCHEMA)  # input schema riêng
  except ValidationError as e:
    return jsonify({
      "error": "validation_failed",
      "field": list(e.path),
      "message": e.message
    }), 400
  book = Book.from_payload(payload)  # cần cài đặt
  db.session.add(book); db.session.commit()
  return book.to_dict(), 201

# 400 kèm field + message — debug dễ
```

✓ Dùng JSON Schema khi

- Làm OpenAPI spec — share với multi-language
- Validate input từ 3rd-party (data scientist tools)
- Generate mock data + test fixture
- Schema reference file (diff dễ)
- Không phụ thuộc Python version

→ Dùng Pydantic khi

- FastAPI project (built-in)
- Cần type hints + IDE support
- Custom validator phức tạp ở mức field
- Internal app, không cần schema file
- Cần parse JSON thành typed object

Lab 1 — models.py: Book + Author

Code rút gọn minh họa ORM; cần bổ sung model và imports.

BookAuthor / Category / Review / User triển khai tương tự

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy.orm import mapped_column, relationship

db = SQLAlchemy()
class Book(db.Model):









  def to_dict(self):
    return {"id": f"book_{self.id}", "title": self.title}

class Author(db.Model):  # cần id primary key
```

test bằng curl *localhost:5000/api/v1/books*

Lab 1 — setup, entities và CRUD routes

Setup

```
mkdir bookstore-api && cd bookstore-api
python -m venv venv
source venv/bin/activate
pip install flask flask-sqlalchemy jsonschema
```

```
bookstore-api/
  app.py
  models.py
  routes/books.py
  schemas/book.json
```

test bằng curl *localhost:5000/api/v1/books*

5 entities & quan hệ

Entity	Fields chính	Quan hệ
Book	title, isbn, price_vnd, stock	N:1 Cat; N:M Author
Author	name, bio	N:M Book
Category	name, slug	1:N Book
Review	rating, book_id, user_id	N:1 Book; N:1 User
User	email, display_name	1:N Order; 1:N Review

Tại sao cần phân trang list endpoint?

❌ Vấn đề: trả toàn bộ collection

Bảng `orders` của một sàn e-commerce có 50 triệu rows. Một endpoint `GET /orders` trả hết sẽ:

- Tiêu thụ RAM và băng thông lớn; phụ thuộc kích thước bản ghi
- Truy vấn lớn có nguy cơ timeout, dù còn tùy kế hoạch thực thi
- Client tải dữ liệu thừa so với nhu cầu hiển thị
- API rate-limit / timeout gateway
- Caching khó tối ưu khi payload và dữ liệu thay đổi

Ví dụ có tài liệu: GitHub REST API sử dụng phân trang và Link header; nhiều API khác cung cấp cursor/page token

Pagination phải giải quyết 4 vấn đề

① Performance
Server không load quá nhiều rows; client không render quá nhiều items

② UX phân trang
Client biết có bao nhiêu trang, trang hiện tại, trang tiếp theo

③ Nhất quán dữ liệu
Item mới thêm vào không làm "skip" hoặc "duplicate" trên các trang

④ Linh hoạt skip khi cần
User có thể nhảy đến trang N, deep-link trang cụ thể

⚠ Lỗi phổ biến không xài pagination

`curl api.com/users | jq '.[0:20]'` — không phải pagination, chỉ là slicing client. Server vẫn trả 50M rows.

Client pagination UI — xác định yêu cầu từ UX

Pagination số
« 1 2 3 ... 47 48 »
Yêu cầu: **tổng trang**, trang hiện tại

Load more
[Load more]
Yêu cầu: **cursor trang tiếp**

Infinite scroll
Kéo xuống tự load
Yêu cầu: **cursor trang tiếp**

Specific page jump
Go to page 47
Yêu cầu: **offset/limit**

Mỗi UX cần một strategy khác nhau — đây là lý do có nhiều pagination strategies, không phải chỉ một cách "đúng".

Các cơ chế pagination và cách biểu diễn liên kết

Mỗi strategy phù hợp với use-case khác nhau. Offset và keyset là kỹ thuật truy vấn; cursor và Link là cách đóng gói/điều hướng.

① Offset / Limit (a.k.a. page number)

```
GET /books?offset=40&limit=20
```

Trả về **20 items** bắt đầu từ vị trí **40**. Dùng SQL `LIMIT 20 OFFSET 40`.

- ✓ Đơn giản, quen thuộc
- ✓ Random page jump OK
- ✗ Chậm khi offset lớn
- ✗ Duplicate khi data thay đổi

② Page number + Link header

```
GET /books?page=3&per_page=20
Link: </books?page=1>; rel="first",
      </books?page=5>; rel="next",
      </books?page=8>; rel="last"
```

Response dùng Link header theo RFC 8288. Body có thể chỉ chứa items, không cần metadata.

- ✓ URL clean
- ✓ HATEOAS-friendly
- ✗ Body mỏng — client cần parse header

③ Keyset / cursor theo khóa sắp xếp

```
GET /books?after_id=42&limit=20
```

Trả 20 items **sau** item có id=42. SQL: `WHERE id > 42 ORDER BY id LIMIT 20`.

- ✓ Nhất quán cao
- ✓ Hiệu năng ổn định ở mọi trang
- ✗ Cursor opaque — không tự tạo được
- ✗ Không jump đến trang N

④ Opaque page token (mã hóa state)

```
GET /books?limit=20
{
  "data": [...20 items...],
  "nextCursor": "eyJpZCI6NDJ9"
}
```

Ví dụ token = base64url(JSON(state)); cần kiểm tra tính toàn vẹn. Client chỉ cần gửi lại `?cursor=...`.

- ✓ Body-based, dễ debug
- ✓ Cursor chứa nhiều state (sort key, filter)

Offset / Limit Pagination — Đơn giản nhưng có giới hạn

3 quy ước đặt tên — chọn 1 và nhất quán

`?offset=40&limit=20` → `?skip=40&take=20` → `?page=3&per_page=20`

Khuyến nghị: `offset` / `limit` (machine-readable, đa dụng).

Triển khai Flask + SQLAlchemy

```python
from sqlalchemy import select, func
DEFAULT, MAX_ = 20, 100

@bp.get("/books")
def list_books():
  o = request.args.get("offset", 0, type=int)
  lim = min(request.args.get(
    "limit", DEFAULT, type=int), MAX_)
  total = db.session.scalar(
    select(func.count(Book.id)))
  rows = db.session.scalars(
    select(Book).order_by(Book.id)
      .offset(o).limit(lim)
  ).all()
  return jsonify({
    "data": [b.to_dict() for b in rows],
    "pagination": {
      "offset": o, "limit": lim,
      "total": total,
      "has_next": o + lim < total
    }
  })
```

⚠ Slow khi offset lớn

```sql
SELECT * FROM books ORDER BY id
LIMIT 20 OFFSET 1000000;
-- Thời gian phụ thuộc index, kế hoạch và dữ liệu
```

Offset lớn có thể phải duyệt nhiều hàng, dù DB vẫn có thể dùng index.

⚠ Duplicate / skip khi data đổi

```
T=0: GET ?offset=20 → items 21–40
T=1: server INSERT book mới ở đầu
T=2: GET ?offset=40 → items 41–60
// dữ liệu dịch vị trí: có thể lặp hoặc bỏ sót
```

Bug khó phát hiện vì client duyệt cùng cách, kết quả *nhảy loạn*.

Khi nào vẫn dùng offset/limit?

① **UI jump-to-page** — admin panel cần "trang 5/47"
② **Dataset tương đối nhỏ hoặc ít biến động**
③ **Total count quan trọng** — "1–20 / 1234"

Dataset lớn/biến động: xem xét keyset/cursor

Cursor pagination — "tiếp tục từ trang X"

Offset — "20 items vị trí 980–1000"

```
id: ... 978    skip 979 rows   [980 981... 999] 1000
```

Cursor — "20 items có id > 979"

```
id: ... 978 979   index seek [980 981 982 ... 999] 1000
```

Cursor cơ bản

```python
@bp.get("/books")
def list_books():
  limit = min(request.args.get(
    "limit", 20, type=int), 100)
  after_id = request.args.get(
    "after_id", type=int)
  q = db.session.query(Book)
  if after_id is not None:
    q = q.filter(Book.id > after_id)
  q = q.order_by(Book.id.asc()).limit(limit + 1)
  rows = q.all()  # +1 trick
  has_more = len(rows) > limit
  rows = rows[:limit]
  next_c = rows[-1].id if rows and has_more else None
  return jsonify({
    "data": [b.to_dict() for b in rows],
    "page_info": {
      "next_cursor": next_c,
      "has_more": has_more
    }
  })
```

Composite cursor — tie-break bằng id

```python
import base64, json

def encode(s):
  return base64.urlsafe_b64encode(
    json.dumps(s).encode()
  ).decode().rstrip("=")

def decode(t):
  pad = "=" * (-len(t) % 4)
  return json.loads(
    base64.urlsafe_b64decode(t + pad))

# Cursor mẫu:
# {"s":"-price", "v":450000, "i":42}
```

Tại sao composite (sort, id)?

2 item cùng sort value → tie. Composite với id đảm bảo deterministic. Cách tie-break này giúp thứ tự xác định.

```
WHERE (price_vnd, id) < (500000, 42)
ORDER BY price_vnd DESC, id DESC LIMIT 20
```

Bảng so sánh: chọn strategy nào cho use-case nào?

Tiêu chí	Offset / Limit	Cursor	Page + Link header	Seek (cursor JSON)
Hiệu năng ở trang đầu	Tùy hệ thống	Tùy hệ thống	Tùy hệ thống	Tùy hệ thống
Hiệu năng ở trang 10,000	Có thể chậm	Phụ thuộc index	Có thể chậm	Phụ thuộc index
Nhất quán khi data đổi	✗ Có duplicate/skip	✓ Rất tốt	✗ Giống offset	✓ Tốt
Có total count	✓ Dễ	✗ Không có sẵn	Không đảm bảo	✗ Cần track riêng
Jump đến trang N	✓ Tự nhiên	✗ Không có	✓ Tự nhiên	✗ Không có
Độ phức tạp code	Rất đơn giản	Trung bình	Trung bình	Phức tạp hơn
Cacheable URL	Phụ thuộc TTL	Phụ thuộc thiết kế	✓ Tốt	Phụ thuộc token

✓ Default recommendation: Cursor-based

Cursor/keyset phù hợp với feed lớn, biến động; không phải mặc định tối ưu cho mọi API.

Lý do: dataset có thể tăng trưởng theo thời gian → API không bị "đứt" khi scale. Cursor decouple logic phân trang khỏi vị trí vật lý của row.

Decision guide

Mobile API / Timeline / Feed: Cursor (giữ trải nghiệm scroll dù có items thay đổi)

Admin panel có UI page-number: Offset/page (UI cần "Trang 5/47")

Collection lớn/biến động: cân nhắc keyset/cursor

Hybrid UI: total count là tính năng tùy chọn, cần thiết kế riêng

E-commerce product list có filter & sort đa chiều: Seek pagination với cursor phức hợp

Lab 2 — parse_sort và filter

Composite (sort, id) cursor — kết hợp filter + sort whitelist.

Phase 1 — parse + filter

```
SORT = {"id": Book.id, "price": Book.price_vnd}

def parse_sort(raw):
  desc = raw.startswith("-")
  key = raw.lstrip("-")
  if key not in SORT:
    return None, "bad_sort", 400
  p = SORT[key]
  cols = [
    p.desc() if desc else p.asc(),
    Book.id.desc() if desc else Book.id.asc()
  ]
  return p, cols, desc

# Gọi trong route:
# p, cols, desc = parse_sort(sort_raw)
# q = select(Book).order_by(*cols)
# if cat_id:
#   q = q.filter(Book.category_id == cat_id)
```

Whitelist sort keys chống SQLi — không bao giờ dùng raw tên cột

Validation cases

- ?sort=price – giá tăng dần
- ?sort=-id – mới nhất
- ?sort=foo → 400
- ?sort=-price,title → chưa hỗ trợ trong lab

Composite key formula

```
WHERE (price_vnd, id) < (500000, 42)
ORDER BY price_vnd DESC, id DESC LIMIT 20
```

Cursor lưu sort value + id

Tie-break = id

⚠ **Chỉ lưu id → bug khi sort dup**

Tách thành 3 helper (parse_sort, apply_cursor, build_response) dễ test đơn lẻ

Cursor pagination cho Books — apply_cursor và build_response

Composite (sort, id) cursor — kết hợp filter + sort whitelist.

Phase 2 — cursor

```
def _enc(s):
  return base64.urlsafe_b64encode(
    json.dumps(s).encode()
  ).decode().rstrip("=")

def apply_cursor(q, raw, sort, p, desc):
  if not raw:
    return q, None, 200
  pad = "=" * (-len(raw) % 4)
  c = json.loads(
    base64.urlsafe_b64decode(raw + pad))
  if c.get("s") != sort:
    return None, "sort_chg", 400
  v, i = c["v"], c["i"]
  op = "<" if desc else ">"
  condition = tuple_(p, Book.id).op(op)(
    (v, i))
  return q.filter(condition), None, 200
```

Phase 2 — return

```
def build_response(items, lim, sort, p, more):
  nxt = None
  if items and more:
    last = items[-1]
    nxt = _enc({
      "s": sort,
      "v": getattr(last, p.key),
      "i": last.id
    })
  return jsonify({
    "data": [b.to_dict() for b in items],
    "page": {"next": nxt, "more": more}
  })
```

Tách thành 3 helper (parse_sort, apply_cursor, build_response) dễ test đơn lẻ

Cursor pagination cho Books — kết hợp route, tests và production tips

Composite (sort, id) cursor — kết hợp filter + sort whitelist.

Combine trong route

```python
@bp.get("")
def list_books():
  raw = request.args.get("cursor")
  lim = min(request.args.get(
    "limit", 20, type=int), 100)
  sort = request.args.get("sort", "-id")
  cat = request.args.get("category", type=int)
  p, cols, desc = parse_sort(sort)
  if p is None:
    return cols, 400
  q = select(Book).order_by(*cols)
  if cat:
    q = q.filter(Book.category_id == cat)
  q, err, code = apply_cursor(q, raw, sort, p, desc)
  if err:
    return {"e": err}, code
  rows = db.session.scalars(q.limit(lim + 1)).all()
  return build_response(
    rows[:lim], lim, sort, p, len(rows) > lim)
```

Tests & production tips

```
curl localhost:5000/api/v1/books
curl "?category=3&sort=-price"
curl "?cursor=eyJ..."
```

① Index (sort, id) tránh filesort

② Cache có TTL phù hợp dữ liệu và SLA

③ Default sort=-id – ưu tiên mới nhất

④ Ký HMAC nếu cần chống sửa cursor

⑤ Ràng buộc cursor với filter/sort/version

Query params pattern — Filter, Sort, Field selection

① Filtering

3 dạng filter chính

Equality: `?category=tech`

Range: `?price_min=100000&price_max=500000`

Multi-value: `?tags=python,api,rest`

Quy tắc đặt tên filter param

```
tên-cột-giá-trị-đơn
tên-cột_min, tên-cột_max
tên-cột_in (list)
```

Tránh: `?filtre=...` (sai chính tả), `?where=...` (lộ SQL).

② Sorting

3 cách phổ biến

A) 1 key + direction: `?sort=-price`

B) Multi-key: `?sort=-price,title`

C) Object style: `?sort_by=price&order=desc`

> **Khuyến nghị: dùng cách A với dấu - cho desc. Chain nhiều sort = list các key cách nhau bằng dấu phẩy. Đây là quy ước đề xuất; API thực tế khác nhau.**

Luôn **whitelist** sort column — không bao giờ truyền raw tên cột vào `order_by()`.

③ Field selection (sparse fieldsets)

Vấn đề

Mobile app chỉ cần `id`, `title`, `cover_url` — không cần `description` 2000 từ.

```
GET /books?fields=id,title,price
```

Khi nào dùng

✓ Response có nhiều field hoặc field kích thước lớn

✓ Đa dạng client — mobile vs desktop khác nhau

✓ List view so với detail view khác nhau

✗ Response nhỏ: có thể không cần

Filter, Sort, Field selection — code minh hoạ

① Flask filtering

```
# Pseudo-code: q, raw và các biến đã được khai báo
filters = {
  "category": Book.category_id == category_id,
  "in_stock": (Book.stock > 0),
  "price_min": Book.price_vnd >= p_min,
  "price_max": Book.price_vnd <= p_max
}
# Chỉ apply filter tồn tại
for k, v in raw.items():
  if k in filters:
    q = q.where(filters[k])
```

② Validate + whitelist

```
ALLOWED = {
  "id", "price_vnd", "title",
  "created_at", "stock"
}
def parse_sort(raw):
  keys = []
  for s in raw.split(","):
    desc = s.startswith("-")
    name = s.lstrip("-")
    if name not in ALLOWED:
      raise ValueError(name)
    col = getattr(Book, name)
    keys.append(col.desc() if desc else col.asc())
  return keys
```

③ Drop non-requested fields

```
def to_dict(book, fields=None):
  full = {
    "id": ..., "title": ...,
    "description": ...,
    "price": ...
  }
  if fields is None:
    return full
  return {
    k: v for k, v in full.items()
    if k in fields
  }
```

5 sai lầm thường gặp & cách tránh

1 Lộ DB schema ra API

Trả `password_hash`, trả tên bảng, internal fields. Khi DB refactor → API break. **Fix:** whitelist field trong serializer; coi serializer là API contract, không phải ORM dump.

2 Quên pagination khi collection lớn

Dev chạy local với 100 records thấy "OK" — production có 10M. **Fix:** từ ngày đầu, mọi collection endpoint phải có `limit`. Default limit + max limit. Test với >10K records trong staging.

3 Cursor sai → trả duplicate / mất items

Cursor chỉ lưu `id` thay vì `(sort_value, id)`. Khi 2 item cùng sort value → bug. **Fix:** luôn composite gồm unique key (id); verify với test có data trùng.

4 N+1 query khi embed related

Embed author cho 100 books → 1 + 100 = 101 query. **Fix:** eager-load (`joinedload` / `selectinload`) hoặc dùng `?expand=author`.

5 Không validate sort/filter input

Truyền raw tên cột vào `order_by()` → SQLi hoặc leak error. **Fix:** luôn whitelist columns; reject nếu unknown.

Best practices từ production (tổng hợp từ Stripe, GitHub, Shopify, Twitter)

① Default limit = 20; max limit = 100

② Cache page 1 của popular sort

③ `Link` header cho next/prev URL (HATEOAS)

④ Cursor không chứa sensitive data

⑤ Đo real performance với queries thật

⑥ Doc rõ behavior khi dataset rỗng

Buổi 5 — Tóm tắt & nội dung tiếp theo

5 điểm cốt lõi cần nhớ

1. **Resource ≠ DB Schema.** Resource là hợp đồng với client, có thể gộp nhiều bảng hoặc drop fields nhạy cảm.
2. **Dùng nested URL khi child phụ thuộc parent, top-level filter khi ngược lại.** N:M có metadata → junction resource.
3. **Cursor pagination là default cho public API.** Offset chỉ khi cần jump-to-page hoặc dataset < 100K.
4. **Cursor composite key phải có id (unique).** Sort key thay đổi → cursor cũ không hợp lệ — phải trả lỗi rõ ràng.
5. **Validate mọi input.** Whitelist sort columns, escape filter values, max limit enforced.

Tài liệu đọc thêm

Bắt buộc

Geewax, "API Design Patterns" — Chapter 3 (Resources) & Chapter 4 (Representation)

Tham khảo

apidocs.co/category/pagination/ — roundup các API lớn

use-the-index-luke.com/sql/partial-results — pagination SQL sâu

Buổi sau chuẩn bị

Buổi 6: Authentication và Authorization — OAuth 2.0 vs JWT, security patterns. Đọc trước: Geewax Chapter 5

Bài tập về nhà (nộp trước buổi 6)

Bài 1 — Resource Design

Chọn một trong 3 domain (*thư viện*, *đặt phòng khách sạn*, *trắc nghiệm online*): vẽ ERD, thiết kế resource set + URL list 12–20 endpoints, viết JSON Schema cho 3 resource chính.

Nộp: PDF (ERD + URL list) + file JSON Schema

Bài 2 — Flask pagination

Hoàn thiện `list_books` ở Lab 1: implement **cả offset và cursor** (chọn qua param `?strategy=...`) + thêm filter `?genre` + sort multi-key.

Nộp: Git repo với tests + Postman collection